

TÀI LIỆU ÔN THI TOÀN DIỆN CƠ SỞ TOÁN HỌC 5

CHƯƠNG AI

Hệ thống hóa Đặc trưng Thuật toán qua Biểu thức Toán học theo Đề cương Học phần

(Biên soạn chuẩn xác theo cấu trúc Đề cương môn học và Giáo trình Russell & Norvig 3rd Edition)

1 CHƯƠNG 1: TỔNG QUAN VỀ TRÍ TUỆ NHÂN TẠO (Introduction)

1.1 1.1. Định nghĩa và khái niệm Trí tuệ nhân tạo (Artificial intelligence definition and concepts)

Định nghĩa AI theo giáo trình Russell & Norvig được cấu trúc hóa dựa trên ma trận hai chiều: Hệ thống suy nghĩ (*Thought processes*) đối lập với Hành vi thực tế (*Behavior*), và Mô phỏng con người (*Human performance*) đối lập với Tính hợp lý tối ưu (*Rationality*).

- *Thinking Humanly*: Mô hình hóa nhận thức của bộ não thông qua các mô phỏng máy tính và thực nghiệm tâm lý học.
- *Acting Humanly*: Đạt được năng lực hành động như con người thông qua việc vượt qua Phép thử Turing (*Turing Test*) - yêu cầu tích hợp các nhánh xử lý ngôn ngữ tự nhiên, biểu diễn tri thức, suy luận tự động, và học máy.
- *Thinking Rationally*: Sử dụng cấu trúc tam đoạn luận và các quy luật logic (*Laws of Thought*) để thiết lập các lập luận đúng đắn tuyệt đối.
- *Acting Rationally (Trọng tâm giáo trình)*: Thiết kế các đại lý thông minh hành động hợp lý (*Rational Agent*) để tối đa hóa hàm hiệu suất kỳ vọng trong môi trường.

1.2 1.2. Lịch sử của Trí tuệ nhân tạo (History of AI)

Sự tiến hóa của AI trải qua các giai đoạn nền tảng: Giai đoạn ập ủ (1943–1955) với mô hình nơ-ron số đầu tiên của McCulloch & Pitts; Sự ra đời của thuật ngữ AI tại hội thảo Dartmouth (1956); Giai đoạn nhiệt huyết và kỳ vọng lớn (1952–1969); Giai đoạn hiện thực phũ phàng và mùa đông AI thứ nhất (1966–1973); Sự bùng nổ của các Hệ chuyên gia dựa trên tri thức (1969–1979); Mùa đông AI thứ hai (1984–1987); và Kỷ nguyên hiện đại của AI dựa trên dữ liệu lớn, toán học xác suất thống kê và học sâu từ năm 1995 đến nay.

1.3 1.3. Các lĩnh vực nghiên cứu và ứng dụng chính (Main research and application areas)

Các nhánh chuyên sâu cốt lõi bao gồm: Thị giác máy tính (*Computer Vision*), Xử lý ngôn ngữ tự nhiên (*NLP*), Hệ thống tự hành và Robot (*Robotics*), Quy hoạch chiến lược tự động (*Automated Planning*), Hệ quản trị tri thức, và Học máy (*Machine Learning*).

1.4 1.4. Thành tựu và những bài toán chưa có lời giải (Achievements and unsolved problems)

- *Thành tựu*: Vượt qua con người trong các trò chơi chiến lược phức tạp (Cờ vua Deep Blue, Cờ vây AlphaGo), xe tự lái cấp độ cao, hệ thống chẩn đoán y tế tự động chuẩn xác và các mô hình ngôn ngữ lớn (LLM).
- *Bài toán chưa giải quyết*: Thách thức về Trí tuệ nhân tạo tổng hợp (*AGI* - Trí tuệ nhân tạo đa dụng cấp chuyên gia), tính minh bạch và khả năng giải thích của mô hình hộp đen (*Explainable AI*), hiện tượng ảo giác dữ liệu, và giới hạn tính toán lý thuyết đối với các bài toán có không gian trạng thái bùng nổ hàm mũ (NP-hard).

2 CHƯƠNG 2: TÌM KIẾM VÀ GIẢI QUYẾT BÀI TOÁN (Search and Problem Solving)

2.1 2.1. Giải quyết bài toán bằng tìm kiếm (Solving problems by searching)

Tìm kiếm trạng thái là quá trình một đại lý thiết lập một chuỗi các hành động tối ưu để di chuyển từ trạng thái hiện tại đến trạng thái đích thông qua việc khám phá một cây hoặc đồ thị tìm kiếm nhân tạo.

2.2 2.2. Phát biểu bài toán (Problem formulation)

Toán học hóa một bài toán tìm kiếm yêu cầu định nghĩa chặt chẽ một bộ gồm 5 thành phần:

1. *Trạng thái khởi đầu (Initial state)*: Trạng thái bắt đầu của tác nhân (Ký hiệu là s_0).
2. *Tập các hành động (Actions)*: Hàm trả về các hành động khả thi tại một trạng thái: $Actions(s)$.
3. *Mô hình chuyển trạng thái (Transition model)*: Hàm xác định trạng thái kết quả khi thực hiện hành động a tại trạng thái s : $Result(s, a)$.
4. *Kiểm tra mục tiêu (Goal test)*: Hàm logic xác định trạng thái hiện tại đã đạt đích hay chưa: $GoalTest(s) \rightarrow \{True, False\}$.
5. *Chi phí đường đi (Path cost)*: Hàm toán học tích lũy chi phí của tuyến đường, tổng hợp từ chi phí từng bước đơn lẻ: $c(s, a, s')$.

2.3 2.3. Tìm kiếm mù / Không có thông tin (Uninformed search)

Đặc trưng của nhóm này là thuật toán duyệt không gian chỉ dựa trên hàm chi phí thực tế $g(n)$ tích lũy từ nút gốc đến nút hiện tại n , hoàn toàn không có ước lượng tương lai.

- **Tìm kiếm theo chiều rộng (BFS)**: Mở rộng các nút nông nhất trước trên cây tìm kiếm thông qua cấu trúc dữ liệu hàng đợi FIFO. Đặc trưng toán học: Độ phức tạp thời gian và không gian đều ở mức lũy thừa $O(b^d)$ (với b là hệ số nhánh, d là độ sâu của nghiệm), đảm bảo tính tối ưu nếu chi phí của mọi bước di chuyển bằng nhau.
- **Tìm kiếm theo chiều sâu (DFS)**: Mở rộng các nút sâu nhất trên nhánh hiện tại trước thông qua cấu trúc ngăn xếp LIFO. Đặc trưng toán học: Tiết kiệm không gian lưu trữ với độ phức tạp không gian tuyến tính chỉ ở mức $O(bm)$ (với m là độ sâu tối đa của cây), nhưng độ phức tạp thời gian cực lớn $O(b^m)$ và không đảm bảo tìm thấy nghiệm tối ưu.
- **Tìm kiếm chi phí đồng nhất (Uniform-Cost Search - UCS)**: Đặc trưng bởi việc mở rộng nút n có chi phí thực tế nhỏ nhất trong hàng đợi ưu tiên. Hàm toán học điều hướng nút đặc trưng:

$$f(n) = g(n) \quad (1)$$

Thuật toán đảm bảo tìm được đường đi ngắn nhất toàn cục trên mọi đồ thị có chi phí cạnh lớn hơn 0.

2.4 2.4. Tìm kiếm có thông tin / Heuristic (Informed search)

Nhóm giải thuật này đưa vào hàm toán học Heuristic $h(n)$ dùng để ước lượng chi phí ngắn nhất từ nút hiện tại n tới trạng thái đích.

- **Tìm kiếm Tham lam (Greedy Best-First Search)**: Đặc trưng bởi hàm toán học đánh giá trạng thái hoàn toàn phụ thuộc vào dự báo tương lai, bỏ qua chi phí quá khứ:

$$f(n) = h(n) \quad (2)$$

Thuật toán di chuyển nhanh về phía đích nhưng rất dễ bị rơi vào bẫy tối ưu cục bộ hoặc vòng lặp vô hạn.

- **Thuật toán Tìm kiếm A^* :** Đặc trưng bằng biểu thức toán học kết hợp cân bằng giữa chi phí thực tế đã đi từ nút gốc $g(n)$ và chi phí ước lượng tương lai $h(n)$:

$$f(n) = g(n) + h(n) \quad (3)$$

- **Cơ sở toán học bảo toàn tính tối ưu của A^* :** Để giải thuật A^* tìm được đường đi ngắn nhất toàn cục, biểu thức toán học $h(n)$ phải thỏa mãn các thuộc tính đặc trưng sau:

- *Tính chấp nhận được (Admissibility - áp dụng cho cây tìm kiếm):* Hàm $h(n)$ không bao giờ được đánh giá cao hơn chi phí thực tế ngắn nhất $h^*(n)$ để đến đích:

$$0 \leq h(n) \leq h^*(n), \quad \forall n \quad (4)$$

- *Tính nhất quán / Đơn điệu (Consistency - áp dụng cho đồ thị tìm kiếm):* Với mọi nút n và nút con n' sinh ra bởi hành động a , hàm $h(n)$ phải tuân thủ bất đẳng thức tam giác không gian:

$$h(n) \leq c(n, a, n') + h(n') \quad (5)$$

2.5. Tìm kiếm cục bộ (Local search)

Áp dụng cho các bài toán mà đường đi không quan trọng, chỉ quan tâm đến trạng thái đích tối ưu. Thuật toán hoạt động bằng cách thay đổi cấu hình trạng thái hiện tại thay vì xây dựng cây tìm kiếm.

- **Tìm kiếm leo đồi (Hill-climbing search):** Duyệt không gian trạng thái theo hướng tăng liên tục của hàm đánh giá chất lượng (hoặc giảm liên tục của hàm chi phí). Chiến lược chọn trạng thái kế tiếp đặc trưng bởi biểu thức toán học tham lam:

$$s_{next} = \arg \max_{s' \in Neighbors(s)} Utility(s') \quad (6)$$

Thuật toán gặp hạn chế lớn khi rơi vào các đỉnh cục bộ (*local maxima*), cao nguyên (*plateaux*) hoặc các gờ sắc nhọn (*ridges*).

- **Thuật toán Luyện kim giả lập (Simulated Annealing):** Kết hợp tìm kiếm leo đồi với bước di chuyển ngẫu nhiên để thoát khỏi tối ưu cục bộ. Đặc trưng toán học của giải thuật dựa trên vật lý nhiệt động lực học: nếu trạng thái con s' xấu hơn trạng thái hiện tại s ($\Delta E = Utility(s') - Utility(s) < 0$), thuật toán không từ bỏ ngay mà chấp nhận chuyển sang trạng thái xấu đó với một xác suất ngẫu nhiên tuân theo phân phối Boltzmann:

$$P(\text{Chấp nhận chuyển trạng thái}) = e^{\frac{\Delta E}{T}} \quad (7)$$

Trong đó, T là tham số nhiệt độ giảm dần theo thời gian theo một lịch trình làm lạnh (*cooling schedule*). Khi T cao, thuật toán chấp nhận nhảy ngẫu nhiên; khi T giảm về mức 0, thuật toán hội tụ về leo đồi thuần túy.

2.6. Dự án và bài tập (Project and exercises)

Thiết lập mã nguồn lập trình mô phỏng các bài toán kinh điển như bài toán Tám quân hậu (8-Queens), bài toán định tuyến bản đồ thành phố thực tế, ứng dụng tìm đường đi tối ưu cho drone an ninh vượt vật cản trong không gian đô thị.

3 CHƯƠNG 3: BIỂU DIỄN TRI THỨC VÀ SUY LUẬN (Knowledge Representation and Reasoning)

3.1. Hệ thống dựa trên tri thức (Knowledge based systems)

Một hệ thống dựa trên tri thức (*Knowledge-based system*) tổ chức mã nguồn AI thành hai phần độc lập: Cơ sở tri thức (*Knowledge Base - KB*) chứa tập hợp câu mệnh đề biểu diễn các sự thật về thế

giới thực bằng một ngôn ngữ trang trọng, và Công cụ suy luận (*Inference Engine*) chứa các thuật toán suy diễn logic độc lập với nội dung tri thức. Đại lý tương tác với hệ thống qua vòng lặp:

$$\text{Tri giác mới} \rightarrow \text{TELL}(KB) \rightarrow \text{Công cụ suy luận xử lý} \rightarrow \text{ASK}(KB) \rightarrow \text{Hành động tối ưu} \quad (8)$$

3.2 Logic mệnh đề (Propositional logic)

- **Cú pháp và Ngữ nghĩa:** Biểu diễn tri thức bằng các biến mệnh đề ký hiệu (P, Q, R) và các liên kết toán học ($\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$).
- **Mô hình hóa toán học của Luật kéo theo logic (Entailment):** Một mô hình m là một phép gán giá trị Đúng/Sai cho tất cả các biến logic. Gọi $M(KB)$ là tập hợp tất cả các mô hình làm cho cơ sở tri thức đúng, và $M(\alpha)$ là tập các mô hình làm cho câu mệnh đề α đúng. Biểu thức toán học thiết lập đặc trưng của hệ suy luận:

$$KB \models \alpha \iff M(KB) \subseteq M(\alpha) \quad (9)$$

Nghĩa là câu α là hệ quả logic của KB nếu và chỉ nếu α luôn đúng trong tất cả các thế giới mà KB đúng.

3.3 Logic bậc nhất (First-order logic)

Mở rộng logic mệnh đề để biểu diễn tri thức một cách tự nhiên và gọn nhẹ hơn nhờ việc đưa vào cấu trúc toán học của: Đối tượng (*Objects* - thực thể trong thế giới), Thuộc tính và Mối quan hệ (*Relations/Predicates*), Hàm (*Functions*), và hai loại Lượng từ toán học ràng buộc biến:

- **Lượng từ toàn thể ($\forall$):** Khẳng định tính chất đúng với mọi đối tượng trong miền xác định, thường đi kèm liên kết kéo theo ($\Rightarrow$).
- **Lượng từ tồn tại ($\exists$):** Khẳng định tính chất đúng với ít nhất một đối tượng trong miền xác định, thường đi kèm liên kết hội ($\wedge$).

3.4 Suy luận trong Logic bậc nhất (Inference using FOL)

Cơ chế suy luận dịch chuyển từ việc lập bảng chân trị bùng nổ không gian trạng thái sang các phép biến đổi trực tiếp trên câu tri thức: Phép thế (*Substitution* $\{x/k\}$), Phép đồng nhất hóa (*Unification*) tìm phép thế θ làm cho hai vị từ phức tạp trở nên giống nhau: $Unify(P(x), P(A)) = \{x/A\}$. Áp dụng các luật suy luận vạn năng như Modus Ponens mở rộng (*Generalized Modus Ponens*).

3.5 Thuật toán phân giải bác bỏ (Resolution refutation)

Thuật toán phân giải bác bỏ là nền tảng của các công cụ suy luận tự động tối ưu nhất, hoạt động dựa trên phương pháp chứng minh phản chứng.

- **Biểu thức toán học thiết lập đặc trưng hệ thống:** Để chứng minh câu α là đúng từ cơ sở tri thức, thuật toán thêm phủ định $\neg\alpha$ vào KB và tiến hành triệt tiêu các bỏ đề đối ngẫu để tìm ra mệnh đề rỗng đại diện cho mâu thuẫn toán học ($\emptyset$):

$$KB \wedge \neg\alpha \models \emptyset \quad (10)$$

- **Quy tắc phân giải tổng quát (Resolution Rule):** Yêu cầu chuyển đổi tất cả các câu tri thức phức tạp về Dạng chuẩn hội (*CNF - Conjunctive Normal Form* - chuỗi các phép hội giữa các mệnh đề disjunction). Biểu thức khử đại số đặc trưng:

$$\frac{l_1 \vee \dots \vee l_i \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_j \vee \dots \vee m_n}{(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)\theta} \quad (11)$$

Trong đó, l_i và m_j là hai bỏ đề đối ngẫu có khả năng đồng nhất hóa thông qua phép thế θ sao cho $l_i\theta = \neg(m_j\theta)$.

3.6 3.6. Các hệ thống suy luận logic (Logic reasoning systems)

Bao gồm các cấu trúc lập trình lập luận tiến (*Forward Chaining*) kích hoạt từ dữ liệu thô đẩy lên mục tiêu, lập luận lùi (*Backward Chaining*) kích hoạt từ giả thuyết đích hạ xuống điều kiện cần, và các hệ thống ngôn ngữ lập trình logic như Prolog.

3.7 3.7. Bài tập (Exercises)

Thực hiện các bài toán chuyển đổi ngôn ngữ tự nhiên sang câu FOL, lập quy trình phân giải CNF để chứng minh các định lý toán học logic.

4 CHƯƠNG 4: SUY LUẬN XÁC SUẤT (Probabilistic Reasoning)

4.1 4.1. Biểu diễn tri thức và suy luận trong điều kiện không chắc chắn (Knowledge representation and reasoning under uncertainty)

Khi môi trường hoạt động có tính chất quan sát được một phần và chứa các yếu tố ngẫu nhiên, các hệ thống logic thuần túy của Chương 3 sẽ bị sụp đổ do không thể biểu diễn mức độ tin cậy của thông tin. Toán học xác suất cung cấp công cụ định lượng hóa độ tin cậy bằng các biến ngẫu nhiên.

4.2 4.2. Quy tắc Bayes và Cơ sở lý thuyết xác suất (Bayes rule and basics of probability)

Đặc trưng toán học của suy luận xác suất dựa trên việc cập nhật niềm tin (*belief*) về một Giả thuyết H khi nhận được bằng chứng thực nghiệm mới E từ hệ thống cảm biến.

- **Biểu thức Quy tắc Bayes tổng quát:**

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)} = \frac{P(E|H) \cdot P(H)}{\sum_i P(E|H_i) \cdot P(H_i)} \quad (12)$$

- **Thành phần tạo đặc trưng thuật toán:**

- $P(H|E)$ là Xác suất hậu nghiệm (*Posterior probability*) - mục tiêu tính toán của AI.
- $P(E|H)$ là Độ hợp lý (*Likelihood*) - xác suất xuất hiện bằng chứng nếu giả thuyết đúng (thu thập dễ dàng qua thống kê thực nghiệm).
- $P(H)$ là Xác suất tiên nghiệm (*Prior probability*) - tỷ lệ khách quan ban đầu của giả thuyết khi chưa có cảm biến.

4.3 4.3. Mạng Bayesian (Bayesian networks)

Một Mạng Bayesian là một cấu trúc cấu hình đồ thị có hướng không chu trình (DAG) $G = (V, E)$, trong đó mỗi nút $X_i \in V$ đại diện cho một biến ngẫu nhiên, và các cạnh E mô tả quan hệ phụ thuộc nhân quả trực tiếp giữa các biến [1].

Biểu thức toán học tạo đặc trưng (Định lý phân rã chuỗi): Xác suất đồng thời đầy đủ của một cấu hình hệ thống gồm n biến ngẫu nhiên được tính bằng tích các xác suất có điều kiện tại từng nút dựa trên tập cha trực tiếp của nó ($Parents(X_i)$):

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | Parents(X_i)) \quad (13)$$

Bản chất tạo đặc trưng hình học: Biểu thức tích chuỗi này bề gãy một bảng phân phối xác suất đồng thời khổng lồ có kích thước hàm mũ 2^n thành tổng các bảng xác suất có điều kiện cục bộ nhỏ gọn có kích thước $\sum 2^{|Parents(X_i)|}$, giúp tối ưu hóa bộ nhớ hệ thống.

4.4 Suy luận trong mạng Bayesian (Inference with Bayesian networks)

Mục tiêu là tính toán phân phối xác suất của một tập các biến truy vấn X_{Query} khi biết giá trị của biến quan sát được $E = e$. Hệ thống áp dụng **Thuật toán loại bỏ biến (Variable Elimination)**.

Biểu thức toán học xử lý đặc trưng: Thuật toán thực hiện gom các tổng $\sum$ đối với các biến ẩn không quan tâm Y và đẩy chúng vào sâu bên trong biểu thức tích chuỗi:

$$P(X_{Query}|E = e) = \alpha \sum_{y_1} \cdots \sum_{y_k} \prod_{i=1}^n P(X_i | Parents(X_i)) \quad (14)$$

Phép biến đổi toán học này phân phối phép nhân đối với phép cộng để tính toán trước các nhân tử trung gian (*factors*), giúp đưa chi phí tính toán từ mức lũy thừa hàm mũ xuống mức tuyến tính tương thích với cấu trúc cây của đồ thị.

4.5 Dự án và bài tập (Project and exercises)

Xây dựng mạng Bayes chẩn đoán lỗi hệ thống phần cứng, hoặc tính toán ma trận xác suất đe dọa của mạng lưới drone địch xâm nhập đô thị dựa trên luồng dữ liệu radar không hoàn hảo.

5 CHƯƠNG 5: HỌC MÁY (Machine Learning)

5.1 Định nghĩa và khái niệm của Học máy (Definition and concepts of machine learning)

Học máy nghiên cứu các thuật toán cho phép hệ thống tự động tối ưu hóa hiệu suất thực thi tác vụ thông qua dữ liệu huấn luyện quá khứ. Phân chia cấu trúc học máy thành: Học có giám sát (*Supervised learning* - học từ cặp dữ liệu đầu vào và nhãn), Học không giám sát (*Unsupervised learning* - tìm cấu trúc ẩn của dữ liệu thô), và Học tăng cường (*Reinforcement learning* - tối ưu hóa hành động qua cơ chế nhận phần thưởng/hình phạt từ môi trường).

5.2 Học cây quyết định (Decision tree learning - Thuật toán ID3)

Thuật toán ID3 xây dựng cây quyết định phân loại bằng cách sử dụng các biểu thức của Lý thuyết thông tin làm bộ lọc Heuristic lựa chọn thuộc tính phân nhánh tối ưu nhất tại mỗi nút.

- **Biểu thức Hàm Entropy (Đo mức độ hỗn loạn thông tin của tập dữ liệu S):** Giả định dữ liệu thuộc c lớp khác nhau, tỷ lệ mẫu dữ liệu thuộc lớp i là p_i :

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i) \quad (15)$$

- **Biểu thức Hàm Lượng thông tin thu nhận (Information Gain) của thuộc tính A :** Đo chỉ số suy giảm độ hỗn loạn của tập dữ liệu S nếu ta tiến hành phân nhánh theo thuộc tính A :

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v) \quad (16)$$

Trong đó, S_v là tập con của S chứa các mẫu mà thuộc tính A mang giá trị v .

- **Hàm quyết định đặc trưng của thuật toán:** Tại mỗi nút cây, thuộc tính tối ưu A^* được trích xuất bằng biểu thức tối đa hóa lợi ích:

$$A^* = \arg \max_A Gain(S, A) \quad (17)$$

5.3 5.3. Phân loại Naive Bayes (Naive Bayes classification)

Thuật toán phân loại Naive Bayes hoạt động dựa trên việc áp dụng ****Giả định độc lập ngây thơ**** (*Naive Independence Assumption*): coi tất cả các thuộc tính mô tả $X = (x_1, x_2, \dots, x_d)$ hoàn toàn độc lập với nhau khi biết trạng thái lớp quyết định C_k .

Biểu thức toán học tạo đặc trưng thuật toán: Sử dụng giả định độc lập để bẻ gãy xác suất đồng thời đa biến thành tích các xác suất đơn biến độc lập:

$$P(C_k|x_1, \dots, x_d) = \frac{P(C_k) \cdot P(x_1, \dots, x_d|C_k)}{P(x_1, \dots, x_d)} \propto P(C_k) \cdot \prod_{j=1}^d P(x_j|C_k) \quad (18)$$

Hàm toán học đưa ra quyết định phân loại cho một mẫu thử mới dựa trên quy tắc tối đa hóa xác suất hậu nghiệm (Maximum A Posteriori - MAP):

$$\hat{y} = \arg \max_{C_k} \left[P(C_k) \cdot \prod_{j=1}^d P(x_j|C_k) \right] \quad (19)$$

Đặc trưng toán học nhân chuỗi $\prod P(x_j|C_k)$ giúp thuật toán giải quyết triệt để hiện tượng sụp đổ số liệu do thiếu mẫu khi số chiều dữ liệu tăng cao (*Curse of Dimensionality*).

5.4 5.4. Thuật toán láng giềng gần nhất (K-nearest neighbor - KNN)

KNN thuộc nhóm thuật toán học thực nghiệm dựa trên mẫu (*Instance-based / Lazy learning*). Thuật toán không có pha huấn luyện tham số mà đặc trưng hình học của nó được cấu hình trực tiếp từ hàm khoảng cách không gian vector.

- **Biểu thức Khoảng cách Minkowski tổng quát (L_p norm):** Dùng để đo đặc độ lân cận giữa mẫu thử x và mẫu huấn luyện y trong không gian đặc trưng d chiều:

$$D_p(x, y) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{\frac{1}{p}} \quad (20)$$

Cấu hình đặc trưng hình học dịch chuyển theo tham số p : Khi $p = 2$, thuật toán mang đặc trưng của khoảng cách đường thẳng vật lý *Euclidean* (L_2); khi $p = 1$, thuật toán mang đặc trưng dịch chuyển mạng lưới ô bàn cờ *Manhattan* (L_1).

- **Hàm toán học quyết định nhãn có trọng số khoảng cách:** Gọi $N_K(x)$ là tập hợp K láng giềng gần nhất của x tìm được từ khoảng cách D_p . Để tối ưu hóa đặc trưng phân chia, các mẫu ở gần hơn phải mang trọng số bầu chọn cao hơn thông qua hàm tỉ lệ nghịch:

$$\hat{y} = \arg \max_v \sum_{y_i \in N_K(x)} w_i \cdot I(v_i = v) \quad \text{với} \quad w_i = \frac{1}{D_p(x, y_i)^2 + \epsilon} \quad (21)$$

Trong đó, $I(\cdot)$ là hàm chỉ thị (bằng 1 nếu biểu thức trong ngoặc đúng, bằng 0 nếu sai) và $\epsilon \rightarrow 0$ là hằng số chống lỗi chia cho mốc số 0. Biểu thức toán học này tạo nên biên phân chia lớp phi tuyến phức tạp dạng đa diện Voronoi (*Voronoi tessellation*).

5.5 5.5. Các thuật toán học máy khác (Other machine learning algorithms)

Giáo trình của Russell & Norvig mở rộng không gian học máy sang mô hình cấu trúc Mạng nơ-ron nhân tạo (*Artificial Neural Networks - ANN*).

- **Mô hình một Nơ-ron toán học (Perceptron):** Đầu vào vector x được tổng hợp tuyến tính bằng tích vô hướng với ma trận trọng số w , cộng sai số chặn b trước khi đi qua hàm kích hoạt phi tuyến (g như Sigmoid, Tanh, hoặc ReLU):

$$h_w(x) = g \left(\sum_{i=1}^d w_i x_i + b \right) = g(w^T x + b) \quad (22)$$

- **Thuật toán Lan truyền ngược (Backpropagation):** Là thuật toán cốt lõi tối ưu hóa mạng nơ-ron sâu. Đặc trưng toán học của giải thuật dựa trên ****Quy tắc đạo hàm chuỗi (Chain Rule của giải tích)**** để tính toán gradient sai số, điều phối luồng cập nhật trọng số Δw_{ji} ngược từ tầng đầu ra về tầng đầu vào nhằm tối thiểu hóa hàm mất mát toàn cục E :

$$\Delta w_{ji} = -\alpha \frac{\partial E}{\partial w_{ji}} = -\alpha \frac{\partial E}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} \quad (23)$$

Trong đó, α là tốc độ học (*learning rate*), z_j là tổng kích hoạt tuyến tính tại nơ-ron j , và $a_j = g(z_j)$ là đầu ra phi tuyến của nơ-ron đó.

COMPREHENSIVE 5-CHAPTER INTRO TO AI EXAM SOURCE

Complete Academic Framework, Mathematical Foundations & Algorithmic Features

(Strictly Mapped with PTIT Course Syllabus Rules and Russell & Norvig Reference Core)

6 CHAPTER 1: INTRODUCTION

6.1 1.1. Artificial intelligence definition and concepts

The textbook by Russell & Norvig categorizes historical AI definitions along a two-dimensional matrix: processes/reasoning vs. behavior, and human-like performance vs. ideal rationality.

- *Thinking Humanly*: Cognitive modeling frameworks that formalize the internal mechanics of the human mind via empirical psychological testing.
- *Acting Humanly*: Passing the operational Turing Test through distinct computational capabilities: NLP, knowledge representation, automated reasoning, and machine learning.
- *Thinking Rationally*: Codifying the “laws of thought” using formal syllogisms and logical notations to build irrefutable argument structures.
- *Acting Rationally (Textbook Core)*: Designing artifacts that act as rational agents, selecting the absolute highest-utility action paths to maximize performance measures under environmental constraints.

6.2 1.2. History of AI

The historical evolution follows distinct operational eras: Gestation (1943–1955) initiating with the binary neural model of McCulloch & Pitts; The formal birth of AI at the Dartmouth conference (1956); Great expectations and initial breakthroughs (1952–1969); The first AI winter triggered by scale limits (1966–1973); The rise of knowledge-based Expert Systems (1969–1979); The second commercial AI winter (1984–1987); and the Modern Era (1995–Present) driven by big data, rigorous probability foundations, and advanced deep learning infrastructures.

6.3 1.3. Main research and application areas

The essential pillars include: Computer Vision, Natural Language Processing (NLP), Autonomous Systems and Robotics, Automated Strategic Planning, Knowledge Engineering, and Machine Learning.

6.4 1.4. Achievements and unsolved problems

- *Achievements*: Superhuman performance in complex strategy board games (Deep Blue chess solver, AlphaGo), high-level autonomous vehicle routing, precision medical diagnostics, and Large Language Models (LLMs).
- *Unsolved Problems*: Realizing Artificial General Intelligence (AGI), resolving black-box uninterpretability via Explainable AI (XAI), mitigating informational hallucination, and overcoming exponential state-space scaling barriers inherent in NP-hard optimization tasks.

7 CHAPTER 2: SEARCH AND PROBLEM SOLVING

7.1 2.1. Solving problems by searching

State-space exploration is a computational routine where an agent computes an optimal sequence of discrete actions connecting an initial state configuration directly to a designated goal state by exploring a search graph.

7.2 2.2. Problem formulation

Formalizing a search problem mathematically necessitates declaring a 5-tuple layout:

1. *Initial state*: The exact atomic state where exploration begins (denoted as s_0).
2. *Actions set*: An operator mapping a state to all valid reachable choices: $Actions(s)$.
3. *Transition model*: A formal transition function returning the result of executing choice a inside state s : $Result(s, a)$.
4. *Goal test*: A mathematical predicate evaluating if a state satisfies goal conditions: $GoalTest(s) \rightarrow \{True, False\}$.
5. *Path cost*: A cumulative weight function aggregating structural step costs across an exploration route: $c(s, a, s')$.

7.3 2.3. Uninformed search

Uninformed search strategies maintain no directional domain knowledge, relying strictly on the actual accumulated cost function $g(n)$ tracking the path from the root vertex to the current node n .

- **Breadth-First Search (BFS)**: Systematically expands the shallowest unexpanded node first, managed via a FIFO queue structure. Time and space horizons scale exponentially to $\mathcal{O}(b^d)$ (where b is the branching factor and d is the goal depth). It guarantees absolute path optimality if all uniform step weights are identical.
- **Depth-First Search (DFS)**: Expands the deepest unexpanded node first, managed via a LIFO stack layout. It optimizes memory consumption with a linear space boundary of $\mathcal{O}(bm)$ (where m represents max depth), but suffers from an exponential time complexity of $\mathcal{O}(b^m)$ and fails to guarantee solution optimality.
- **Uniform-Cost Search (UCS)**: Guided mathematically by expanding the lowest-cost unexpanded node inside a priority queue. The node evaluation function is defined as:

$$f(n) = g(n) \tag{24}$$

UCS guarantees global cost minimality across any graph where individual edge costs strictly exceed zero.

7.4 2.4. Informed search

Informed search models inject domain-specific hints through a heuristic function $h(n)$ that estimates the minimal path cost remaining from current node n to the destination goal state.

- **Greedy Best-First Search**: Node expansion priority relies exclusively on goal proximity projections, completely discarding historical path weights:

$$f(n) = h(n) \tag{25}$$

While computationally swift, it is highly vulnerable to local optima traps and infinite loops.

- **A* Search Engine:** Unifies accumulated historical path execution costs $g(n)$ with projected remaining path costs $h(n)$ to guide exploration:

$$f(n) = g(n) + h(n) \quad (26)$$

- **Mathematical Conditions for A* Optimality:** To mathematically guarantee that the A* engine isolates the global shortest path, the heuristic function $h(n)$ must obey formal predicates:

- *Admissibility (For Tree Search):* The function $h(n)$ must never overestimate the true optimal path cost $h^*(n)$ required to reach the target goal:

$$0 \leq h(n) \leq h^*(n), \quad \forall n \quad (27)$$

- *Consistency / Monotonicity (For Graph Search):* For any node n generating a successor n' via action a , the heuristic must satisfy the formal triangle inequality constraint:

$$h(n) \leq c(n, a, n') + h(n') \quad (28)$$

7.5 2.5. Local search

Local search models operate in configurations where the exploration path is completely irrelevant, and the objective centers on maximizing or minimizing a target state's score profile.

- **Hill-Climbing Search:** Navigates the state space strictly along the direction of increasing utility metrics. The greedy successor selection strategy is formulated as:

$$s_{next} = \arg \max_{s' \in Neighbors(s)} Utility(s') \quad (29)$$

This strategy frequently fails when encountering local maxima, flat plateaux, or sharp ridges.

- **Simulated Annealing Algorithm:** Blends greedy hill-climbing with randomized adjustments to escape local traps, drawing its mathematical mechanics from statistical thermodynamics. If a successor state s' degrades current performance ($\Delta E = Utility(s') - Utility(s) < 0$), the algorithm transitions to it probabilistically based on the Boltzmann distribution:

$$P(\text{Accept State Transition}) = e^{\frac{\Delta E}{T}} \quad (30)$$

Where T marks a temperature schedule parameter that monotonically decreases over time. At high T levels, wide random state jumps are permitted; as $T \rightarrow 0$, the framework converges into a pure greedy hill-climbing optimization.

7.6 2.6. Project and exercises

Coding assignments simulate classic combinatorial setups such as the 8-Queens constraint problem, real-world street network routing optimization, and tactical drone path planning avoiding high-rise urban structures.

8 CHAPTER 3: KNOWLEDGE REPRESENTATION AND REASONING

8.1 3.1. Knowledge based systems

A knowledge-based system splits AI engineering into two distinct decoupled blocks: the Knowledge Base (KB), a formal repository storing structured sentences describing real-world assertions using symbolic syntax, and the Inference Engine, which implements execution algorithms to derive new properties independently of domain content. The interaction cycle follows the primary primitive operations:

$$\text{New Percept} \rightarrow \text{TELL}(KB) \rightarrow \text{Inference Processing} \rightarrow \text{ASK}(KB) \rightarrow \text{Optimal Action} \quad (31)$$

8.2 3.2. Propositional logic

- **Syntax and Semantics:** Represents factual concepts via symbolic propositional variables (P, Q, R) unified by mathematical logical connectives ($\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$).
- **Mathematical Modeling of Entailment:** A model m is a formal mapping assigning static truth values (True/False) to variables. Let $M(KB)$ be the extensive set of all models satisfying KB , and $M(\alpha)$ represent models satisfying sentence α . The structural feature of logical entailment is formalized as:

$$KB \models \alpha \iff M(KB) \subseteq M(\alpha) \quad (32)$$

This equation states that α is a necessary logical consequence of KB if and only if α holds true in every possible world model configuration where KB is true.

8.3 3.3. First-order logic

Enhances the expressive power of propositional logic by decomposing real-world knowledge into structured sets of: Objects (entities in the domain), Relations and Predicates (properties connecting objects), Functions, and two dedicated mathematical Quantifiers:

- **Universal Quantifier ($\forall$):** Declares that a specific predicate holds true for every single object instance in the domain, typically paired with the implication connector ($\Rightarrow$).
- **Existential Quantifier ($\exists$):** Declares that a specific predicate holds true for at least one object instance in the domain, typically paired with the conjunction connector ($\wedge$).

8.4 3.4. Inference using FOL

Deduction translates from brute-force truth table model matching to symbolic mechanical manipulations: applying substitution syntax ($\{x/k\}$) and computing formal ****Unification**** rules to isolate a substitution vector θ that aligns variable predicate structures: $Unify(P(x), P(A)) = \{x/A\}$. This unifier powers generalized inference engines like Generalized Modus Ponens.

8.5 3.5. Resolution refutation

The resolution refutation framework serves as the structural foundation for modern automated deduction systems, executing proofs through contradiction.

- **Mathematical Proof Framework:** To prove that a target hypothesis α logically follows from the system database, the engine appends the negated sentence $\neg\alpha$ directly to KB and iteratively applies resolution rules to isolate an empty clause ($\emptyset$), which represents a logical contradiction:

$$KB \wedge \neg\alpha \models \emptyset \quad (33)$$

- **Generalized Resolution Calculation Rule:** Requires translating all logical sentences into Conjunctive Normal Form (CNF - a conjunction of disjunctive clauses). The mathematical literal elimination feature is formulated as:

$$\frac{l_1 \vee \dots \vee l_i \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_j \vee \dots \vee m_n}{(l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)\theta} \quad (34)$$

Where complementary literals l_i and m_j are unified via substitution vector θ such that $l_i\theta = \neg(m_j\theta)$, causing them to systematically cancel out.

8.6 3.6. Logic reasoning systems

Includes production architectures utilizing Forward Chaining (data-driven models moving from basic assertions up to goals), Backward Chaining (goal-driven sub-query tracking to verify prerequisites), and logic programming environments like Prolog.

8.7 3.7. Exercises

Parsing natural language assertions into structured FOL statements and building resolution refutation trees to mechanically prove formal mathematical theorems.

9 CHAPTER 4: PROBABILISTIC REASONING

9.1 4.1. Knowledge representation and reasoning under uncertainty

When operating in partially observable and stochastically volatile environments, static logic databases fail due to their inability to quantify partial degrees of belief. Probability theory resolves this by modeling operational states through random variables.

9.2 4.2. Bayes rule and basics of probability

The core feature of probabilistic reasoning centers on updating the calculated belief in a Hypothesis H when receiving a new empirical telemetry stream E from hardware sensors.

- **Generalized Bayes' Rule Formulation:**

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)} = \frac{P(E|H) \cdot P(H)}{\sum_i P(E|H_i) \cdot P(H_i)} \quad (35)$$

- **Algorithmic Characterization Components:**

- $P(H|E)$ represents the Conditional Posterior Probability, the ultimate objective of the AI tracking backend.
- $P(E|H)$ tracks the Likelihood, the probability that the evidence occurs given the hypothesis is true (extracted via statistical history).
- $P(H)$ defines the Objective Prior Probability, the base baseline occurrence rate of the threat before activating sensor streams.

9.3 4.3. Bayesian networks

A Bayesian Network is a structured Directed Acyclic Graph (DAG) $G = (V, E)$ where vertices V map distinct random variables and directed edges E explicitly quantify direct causal dependencies across fields [1].

Mathematical Joint Distribution Factoring (Chain Rule): The complete joint probability distribution over the global space of n random variables is factored into compact products of localized conditional states governed strictly by immediate parent nodes ($Parents(X_i)$):

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | Parents(X_i)) \quad (36)$$

Algorithmic Feature Compression: This chain product systematically compresses a massive joint distribution table scaling exponentially at 2^n down to compact localized tables scaling linearly at $\sum 2^{|Parents(X_i)|}$.

9.4 4.4. Inference with Bayesian networks

The target objective is to compute the exact probability profile of a set of query variables X_{Query} given observed historical evidence states $E = e$. The system implements the **Variable Elimination (VE)** engine.

Mathematical Core Filtering Equation: VE collects summation operators over hidden unobserved variables Y and distributes them deep inside the product chain:

$$P(X_{Query} | E = e) = \alpha \sum_{y_1} \dots \sum_{y_k} \prod_{i=1}^n P(X_i | Parents(X_i)) \quad (37)$$

This mathematical transformation distributes multiplications over additions to pre-compute intermediate factors, converting exponential computational costs into a linear space bounded by the graph's tree-width.

9.5 4.5. Project and exercises

Engineering Bayesian diagnostic systems for hardware fault localization, or resolving live threat matrix assessments for drone swarm tracking using noisy radar observations.

10 CHAPTER 5: MACHINE LEARNING

10.1 5.1. Definition and concepts of machine learning

Machine learning designs algorithms that allow software systems to optimize task execution performance autonomously through training data. Structural classification isolates Supervised Learning (mapping inputs directly to labeled fields), Unsupervised Learning (discovering latent patterns in unlabeled data), and Reinforcement Learning (optimizing dynamic behaviors via rewards and penalties).

10.2 5.2. Decision tree learning

The ID3 algorithm compiles descriptive categorization models by leveraging information-theoretic equations as a heuristic mechanism to select the absolute highest-utility splitting variable at each branch node.

- **Information Entropy Expression (Data Impurity Measurement):** For dataset S spanning c target classes where p_i is the density of class i :

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i) \quad (38)$$

- **Information Gain Expression (Attribute A Splitting Value):** Computes the expected reduction in entropy achieved by partitioning dataset S along attribute A :

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v) \quad (39)$$

Where S_v isolates the subset of S where attribute A holds value v .

- **Core Heuristic Selector Equation:** Branch node creation is governed by maximizing information extraction:

$$A^* = \arg \max_A Gain(S, A) \quad (40)$$

10.3 5.3. Naive Bayes classification

The Naive Bayes engine constructs classification decisions by enforcing the ****Naive Independence Assumption****, asserting that all descriptive attributes $X = (x_1, x_2, \dots, x_d)$ are conditionally independent given the targeted class variable C_k .

Algorithmic Characterization Formula: Imposing independence factors multi-variable distributions into basic atomic chain products:

$$P(C_k | x_1, \dots, x_d) = \frac{P(C_k) \cdot P(x_1, \dots, x_d | C_k)}{P(x_1, \dots, x_d)} \propto P(C_k) \cdot \prod_{j=1}^d P(x_j | C_k) \quad (41)$$

The final classification is isolated by selecting the class that maximizes the structural Maximum A Posteriori (MAP) criteria:

$$\hat{y} = \arg \max_{C_k} \left[P(C_k) \cdot \prod_{j=1}^d P(x_j | C_k) \right] \quad (42)$$

The mathematical chain product $\prod P(x_j|C_k)$ allows the model to efficiently compute boundaries over sparse datasets, entirely bypassing the curse of dimensionality.

10.4 5.4. K-nearest neighbor

KNN is a non-parametric, instance-based lazy learning framework. The model skips parametric training entirely, meaning its decision boundaries are defined explicitly by spatial geometry calculations over vector fields.

- **Generalized Minkowski Metric Expression (L_p norm):** Quantifies spatial proximity configurations between a query sample x and training record y across a d -dimensional space:

$$D_p(x, y) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{\frac{1}{p}} \quad (43)$$

Modulating parameter p modifies geometric features: setting $p = 2$ yields linear *Euclidean* metrics (L_2 norm); setting $p = 1$ maps grid-like *Manhattan* distance pathways (L_1 norm).

- **Distance-Weighted Label Assignment Selector:** Let $N_K(x)$ represent the isolated set of K nearest neighbors computed via metric D_p . To optimize classification boundaries, closer records are assigned higher voting weights via an inverse relation:

$$\hat{y} = \arg \max_v \sum_{y_i \in N_K(x)} w_i \cdot I(v_i = v) \quad \text{where} \quad w_i = \frac{1}{D_p(x, y_i)^2 + \epsilon} \quad (44)$$

Where $I(\cdot)$ is the standard binary indicator function and $\epsilon \rightarrow 0$ is a stabilizing constant preventing division by zero. This mathematical formulation yields highly adaptive, non-linear local decision fields (*Voronoi tessellation*).

10.5 5.5. Other machine learning algorithms

The textbook by Russell & Norvig extends machine learning into structures modeling biological neural networks through Artificial Neural Networks (ANNs).

- **Mathematical Node Profiling (Perceptron):** An input vector x undergoes a linear combination via a dot product with weight matrix w , adding a bias offset b , before processing through a non-linear activation operator (g e.g., Sigmoid, Tanh, or ReLU):

$$h_w(x) = g \left(\sum_{i=1}^d w_i x_i + b \right) = g(w^T x + b) \quad (45)$$

- **The Backpropagation Optimization Solver:** The core optimization engine driving deep architectures. The algorithm leverages the calculus **Chain Rule** to compute error gradients, distributing weight corrections Δw_{ji} backward from the output layer to hidden layers to minimize global structural loss values (E):

$$\Delta w_{ji} = -\alpha \frac{\partial E}{\partial w_{ji}} = -\alpha \frac{\partial E}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} \quad (46)$$

Where α dictates the network learning rate, z_j defines the net linear activation at node j , and $a_j = g(z_j)$ marks the final activated output profile of that node.

References

- [1] Dolev Mutzari, Tonmoay Deb, Cristian Molinaro, Andrea Pugliese, V.S. Subrahmanian, and Sarit Kraus. Defending a city from multi-drone attacks: A sequential stackelberg security games approach. *Artificial Intelligence*, 349:104425, 2025.